

Selection control structures

1. What is a control structure in programming?
 - a. **To define the flow of execution in a program**
 - b. To store data in memory
 - c. To represent graphical output
 - d. To connect external devices

Explanation: Control structures help determine the flow of program execution.

2. What is the purpose of conditional statements in a program?
 - a. To repeat a block of code
 - b. **To execute code based on a condition**
 - c. To store large amounts of data
 - d. To display graphical elements

Explanation: Conditional statements execute specific blocks of code depending on whether conditions are true or false.

3. In an "if" statement, when does the code block execute?
 - a. When the condition is false
 - b. **When the condition is true**
 - c. Regardless of the condition
 - d. Only when a loop is running

Explanation: The code block within an "if" statement executes only when the condition evaluates to true.

4. How is an "if-else" statement different from an "if" statement?
 - a. It doesn't allow conditions
 - b. **It has an additional block for false conditions**
 - c. It runs all blocks of code simultaneously
 - d. It can only check numerical values

Explanation: An "if-else" statement includes an additional block for execution when the condition is false.

5. What is a nested "if" statement?
 - a. An "if" statement with no condition
 - b. **An "if" statement inside another "if" statement**
 - c. A condition without an "else" block
 - d. A switch-case inside an "if" statement

Explanation: A nested "if" statement is an "if" statement inside another "if" statement.

6. In which situation would you use a nested "if" statement?
- a. **When checking multiple conditions that depend on each other**
 - b. When a single condition is sufficient
 - c. When a loop is required
 - d. When a variable is undefined

Explanation: A nested "if" statement is used when multiple conditions need to be checked, with one condition depending on another.

7. What is the purpose of the "else" keyword?
- a. To define a condition
 - b. To repeat the code block
 - c. **To provide an alternative action when a condition is false**
 - d. To initialize variables

Explanation: The "else" keyword provides an alternative action when the condition in an "if" statement is false.

8. How many "else" blocks can an "if" statement have?
- a. None
 - b. **One**
 - c. Two
 - d. Multiple

Explanation: An "if" statement can only have one "else" block, but it can have multiple "if" or "else if" blocks.

9. Can an "if-else" structure have multiple "else" blocks?
- a. Yes
 - b. **No**
 - c. It depends on the language
 - d. Only with nested "if" statements

Explanation: An "if-else" structure can only have one "else" block, though multiple "else if" blocks can be used.

10. What is a switch-case statement used for?
- a. To store data
 - b. **To evaluate multiple conditions based on a single variable**
 - c. To repeat code
 - d. To define a function

Explanation: A switch-case statement is used to evaluate multiple conditions based on the value of a single variable.

11. How is a switch-case different from an "if-else" structure?

- a. **Switch-case is more efficient for checking a single variable with many possible values**
- b. Switch-case can only be used with numbers
- c. Switch-case is more flexible than "if-else"
- d. Switch-case does not need a condition

Explanation: Switch-case is more efficient for handling multiple values of a single variable, whereas "if-else" can handle more complex conditions.

12. What is the purpose of the "break" statement in switch-case?

- a. **To break the switch-case structure and continue with the rest of the program**
- b. To exit a loop
- c. To stop the program execution
- d. To terminate a function

Explanation: The "break" statement is used to exit a switch-case structure and continue with the program.

13. What happens if the "break" statement is omitted in a switch-case?

- a. The code will skip all cases
- b. The program will terminate
- c. **The cases will "fall through" and execute one after another**
- d. The program will crash

Explanation: If the "break" statement is omitted, the cases in the switch-case structure will "fall through," meaning all subsequent cases will execute until a break is encountered.

14. What is the purpose of the "default" case in a switch-case structure?

- a. **To handle cases that don't match any of the specified "case" values**
- b. To terminate the program
- c. To store the program's output
- d. To repeat the case execution

Explanation: The "default" case handles situations where no other cases match the value in the switch expression.

15. What type of data is typically used in switch-case conditions?

- a. Strings
- b. Booleans
- c. **Integers or characters**
- d. Arrays

Explanation: Switch-case conditions typically work with integers or characters.

16. Which type of structure is used to handle a menu-driven program?

a. **Switch-case**

b. For loop

c. While loop

d. Recursion

Explanation: A switch-case structure is commonly used to handle menu-driven programs, where the user selects an option.

17. What will be the output of the following code?

```
int x = 5;
if (x > 3)
if (x < 10)
    printf("A");
else
    printf("B");
```

a. **A**

b. B

c. A followed by B

d. No output

Explanation: The first **if** condition (**x > 3**) evaluates to **true**, so the control enters the nested **if**. The condition **x < 10** is also **true**, so **A** is printed. The **else** is ignored because its corresponding **if** is not **false**.

18. What will be the output of the following code?

```
int x = 2;

switch (x) {

    case 1: printf("One");

    case 2: printf("Two");

    case 3: printf("Three");

    default: printf("Default");

}
```

- a. Two
- b. Two Three Default**
- c. Default
- d. Compilation error

Explanation: The `switch` statement does not break after each case, so it executes all cases from the matched one (`case 2`) to the `default`.

19. Which of the following scenarios is best handled by a **nested if-else** structure?

- a. Checking if a number is even or odd.
- b. Validating a password and checking if it has special characters.**
- c. Iterating over a list of numbers.
- d. Printing the first 10 multiples of a number.

Explanation: A **nested if-else** is suitable when multiple related conditions must be checked sequentially.

20. What will the following code output?

```
int x = 4;
switch (x) {
    case 1: printf("One"); break;
    case 2: printf("Two"); break;
    case 3: printf("Three"); break;
    default: printf("None");
}
```

- a. One
- b. Two
- c. Three
- d. **None**

Explanation: Since there is no case for $x = 4$, the control goes to the `default` block, and "None" is printed.